

Data Access and Storage with Python

Flat Files (CSV), Diverse Formats (Excel, JSON, HTML, PDF), SQLite & MongoDB

Dr. Minh Duc Vu

School of Data Science & Artificial Intelligence – National Economics University (NEU)

September 2026

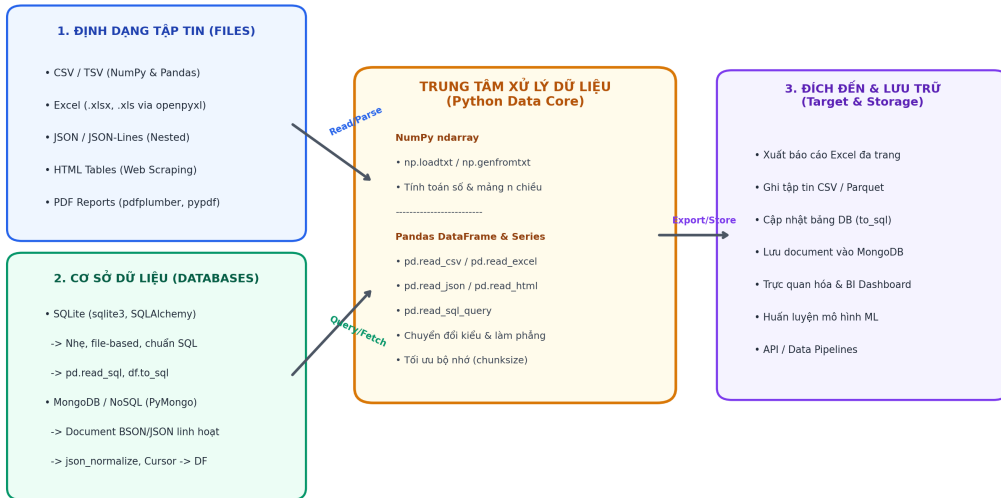
Learning Objectives (CLO 5.4)

Upon completing this module, learners will be able to:

- ▶ **CSV Ingestion & Export:** Master `np.loadtxt`, `np.genfromtxt`, and advanced `pd.read_csv()` features (`chunksize`, `encoding`, `parse_dates`).
- ▶ **Diverse File Formats:** Process multi-sheet Excel workbooks (`openpyxl`), unpack nested JSON trees (`json_normalize`), scrape HTML tables, and extract PDF records.
- ▶ **Relational Databases:** Connect to SQLite via `sqlite3` & `SQLAlchemy`, execute parameterized queries, and bridge two-way transfers with Pandas (`read_sql_query`, `to_sql`).
- ▶ **NoSQL Document Stores:** Connect to MongoDB via `pymongo`, execute CRUD operations, and normalize BSON `ObjectId` records into DataFrames.
- ▶ **End-to-End Pipeline:** Architect an automated multimodal data pipeline for business intelligence reporting.

Python Data Access & Storage Ecosystem

HỆ SINH THÁI TRUY XUẤT VÀ LƯU TRỮ DỮ LIỆU TRONG PYTHON (Python Data Access & Storage Ecosystem)



Reading CSV Data with NumPy

NumPy provides two primary functions for text-based matrix loading:

`np.loadtxt()`

Ideal for clean, pure numerical matrices with no missing values.

```
import numpy as np
# Ingest homogeneous numeric matrix
arr = np.loadtxt("matrix.csv",
                 delimiter=";",
                 skiprows=1)
print(arr.shape)
```

`np.genfromtxt()`

Robust parsing with missing value imputation.

```
# Replace missing cells with 0.0
arr_gen = np.genfromtxt(
    "numeric_sample.csv",
    delimiter=";",
    skip_header=1,
    filling_values=0.0
)
```

Export array to CSV: `np.savetxt("out.csv", arr, delimiter=";", fmt="%0.2f")`.

Essential Parameters of Pandas read_csv()

```
import pandas as pd

df = pd.read_csv(
    "data/customers.csv",
    sep=";",
    index_col="customer_id",
    usecols=["customer_id", "full_name", "signup_date", "total_spent"],
    dtype={"customer_id": str},
    parse_dates=["signup_date"],
    na_values=["NA", "null", "-"],
    encoding="utf-8"
)
# Column delimiter character (default ',')
# Designate primary index column
# Save RAM
# Explicit type enforcement
# Parse ISO date strings to datetime64
# Sentinel missing values
# Universal text encoding
```

Production Export Tip

Use `df.to_csv("clean.csv", index=False, encoding="utf-8-sig")`. The `utf-8-sig` parameter includes a Byte Order Mark (BOM) enabling Microsoft Excel to automatically render UTF-8 characters properly.

Out-of-Core Processing with chunksize

When datasets exceed available physical RAM, loading the entire file raises a fatal **MemoryError**. Chunk streaming processes records iteratively:

```
import pandas as pd

# Streaming iterator loading 100,000 rows per batch
chunk_iter = pd.read_csv("large_transactions.csv", chunksize=100000,
                        usecols=["store_id", "revenue"])

store_totals = {}
for i, chunk in enumerate(chunk_iter):
    # Process each batch independently in RAM
    grouped = chunk.groupby("store_id")["revenue"].sum()
    for sid, rev in grouped.items():
        store_totals[sid] = store_totals.get(sid, 0) + rev
    print(f"-> Finished chunk #{i+1}")

print("Total aggregated revenue:", store_totals)
```

Working with Excel Workbooks (openpyxl)

Reading Multiple Worksheets

```
# Ingest specific worksheet
df = pd.read_excel("products.xlsx",
                  sheet_name="Products",
                  engine="openpyxl")

# Read ALL sheets into a dict of DFs
all_sheets = pd.read_excel(
    "products.xlsx",
    sheet_name=None,
    engine="openpyxl"
)
# all_sheets["Categories"] is a DF
```

Multi-Sheet Excel Export

```
# Write multi-sheet via ExcelWriter
with pd.ExcelWriter("report.xlsx",
                   engine="openpyxl") as w:
    df_prod.to_excel(w,
                    sheet_name="Products",
                    index=False)
    df_kpi.to_excel(w,
                   sheet_name="Summary_KPI",
                   index=False)
```

Flattening Nested JSON Hierarchies (json_normalize)

RESTful API payloads commonly arrive with complex multi-level nesting (**Sub-dictionaries & Arrays**).

```
import json, pandas as pd

with open("ecommerce_transactions.json", "r", encoding="utf-8") as f:
    orders = json.load(f)

# Unpack item arrays while propagating top-level order metadata
df_flat = pd.json_normalize(
    orders,
    record_path=["items"],
    meta=[
        ["order_id", "order_date",
         "customer", "customer_id"],
        ["customer", "name"],
        ["customer", "city"]
    ],
    # Explode rows per item element
    # Order-level attributes
    # Nested sub-dictionary keys
)
print(df_flat.head(3))
```

Web Table Scrapping & PDF Tabular Extraction

HTML Table Parsing (read_html)

```
# Parse all <table> DOM elements
tables = pd.read_html(
    "financial_quotes.html",
    match="VN-Index" # Filter by keyword
)
df_stocks = tables[0]
print(df_stocks.head())
```

PDF Extraction (pdfplumber)

```
import pdfplumber

with pdfplumber.open("invoice.pdf") as pdf:
    for page in pdf.pages:
        tables = page.extract_tables()
        for t in tables:
            df = pd.DataFrame(t[1:],
                              columns=t[0])

            print(df)
```

Relational (SQLite) vs Document NoSQL (MongoDB)

CƠ SỞ DỮ LIỆU QUAN HỆ (SQLITE)

Mô hình Bảng - Dòng - Cột (Schema cố định, ACID)

Bảng: Customers

customer_id	name	city	total_spent
101	Nguyễn Văn A	Hà Nội	2,450,000
102	Trần Thị B	Đà Nẵng	1,800,000
103	Lê Hoàng C	TP. HCM	5,120,000

Lệnh Python thường dùng:

- `conn = sqlite3.connect('company.db')`
- `df = pd.read_sql_query('SELECT * FROM Customers', conn)`
- `df.to_sql('Customers', conn, if_exists='append', index=False)`

CƠ SỞ DỮ LIỆU NOSQL (MONGODB)

Mô hình Document JSON/BSON (Schema động, lỏng nhau)

Collection: orders (Document mẫu)

```
{
  "_id": ObjectId("64a..."),
  "order_id": "ORD-9981",
  "customer": { "name": "Lê Hoàng C", "city": "TP. HCM" },
  "items": [
    { "sku": "P01", "qty": 2, "price": 1200000 },
    { "sku": "P05", "qty": 1, "price": 2720000 }
  ],
  "status": "completed"
}
```

Lệnh PyMongo thường dùng:

- `client = pymongo.MongoClient('mongodb://localhost:27017')`
- `docs = list(client.db.orders.find({'status': 'completed'}))`
- `df = pd.json_normalize(docs) # Làm phẳng dữ liệu l[ô]ng`

Relational Databases: SQLite & Pandas

```
import sqlite3, pandas as pd

conn = sqlite3.connect("data/company.db")

# 1. Direct multi-table JOIN query into Pandas DataFrame
query = (
    "SELECT e.emp_id, e.full_name, d.dept_name, SUM(s.revenue) AS total_revenue "
    "FROM employees e "
    "JOIN departments d ON e.dept_id = d.dept_id "
    "JOIN monthly_sales s ON e.emp_id = s.emp_id "
    "GROUP BY e.emp_id, e.full_name, d.dept_name "
    "ORDER BY total_revenue DESC;"
)
df_sales = pd.read_sql_query(query, conn)

# 2. Persist transformed results back to SQLite
df_sales.to_sql("top_sales_summary", conn, if_exists="replace", index=False)
conn.close()
```

Document NoSQL Database: MongoDB (pymongo)

```
from pymongo import MongoClient
import pandas as pd

# Connect to database instance
client = MongoClient("mongodb://localhost:27017/")
db = client["retail_store_db"]
orders_col = db["customer_orders"]

# 1. Flexible document insertion
orders_col.insert_one({"order_id": "ORD-101", "items": ["P01", "P02"], "amount": 250.00})

# 2. Filter collection stream and normalize to DataFrame
cursor = orders_col.find({"amount": {"$gte": 100.00}})
df_orders = pd.json_normalize(list(cursor))

# Cleanse special BSON ObjectId identifier
if "_id" in df_orders.columns:
    df_orders["_id"] = df_orders["_id"].astype(str)
```

Quick Reference Matrix for Data I/O

Format	Read Function	Write Function	Core Dependency
CSV / TSV	<code>pd.read_csv()</code>	<code>df.to_csv()</code>	Pandas (built-in)
Excel	<code>pd.read_excel()</code>	<code>df.to_excel()</code>	openpyxl
JSON	<code>pd.json_normalize()</code>	<code>df.to_json()</code>	json, Pandas
HTML	<code>pd.read_html()</code>	<code>df.to_html()</code>	lxml, bs4
PDF	<code>page.extract_tables()</code>	-	pdfplumber
SQLite	<code>pd.read_sql_query()</code>	<code>df.to_sql()</code>	sqlite3, SQLAlchemy
MongoDB	<code>col.find()</code> -> DF	<code>col.insert_many()</code>	pymongo

Production Security & Performance Principles

1. Always use parameterized queries (?) to protect against SQL Injection. 2. Manage file and connection handles with `with` statements. 3. Restrict columns with `usecols` and stream large tables via `chunksize`.

Summary & Discussion Questions

Happy Coding & Effective Analytics!

Discussion Questions for Lab Sessions

- 1 When should e-commerce orders be stored as NoSQL Documents rather than normalized relational tables?
- 2 How would you architect an automated nightly cron pipeline loading Web API JSON streams into SQLite?

Contact: Dr. Minh Duc Vu (minhvd@neu.edu.vn)